Modeling Human Activity States Using Hidden Markov Models

Group21 | Accelerometer& Gyroscope | Walking · Standing · Still · Jumping

Users: Rob & Antony | Window: 1 s non-overlapping | SEED: 42 | 8 HMMs trained

Team & Task Allocation

Member	Phone	Sampling Rate	Primary Tasks
Rob	Iphone 15 Pro	100 Hz	Data collection (25 files), HMM class, forward/backward pass, Baum-Welch, Viterbi
Antony	SamSung	100 Hz	Data collection (25 files), feature extraction, normalisation, visualisations, report

Table 1: Team composition and task breakdown. GitHub commit history reflects approximately 50/50 split.

1. Background & Motivation

HumanActivityRecognition (HAR)from inertial sensor data is foundational to wearable health monitoring, smart rehabilitation, and assistive technology. In clinical physiotherapy, patients are typically prescribed structured movement regimens specific durations of walking, standing, still and Jumping that are difficult to monitor remotely. Our group's use case is **outpatient physiotherapy compliance monitoring**: a smartphone worn at waist level continuously logs activity states, enabling therapists to verify that prescribed activity durations are met without requiring clinic visits. Hidden Markov Models are particularly suited to this problem because human activities are temporally structured they persist for seconds or minutes before transitioning and the true activity is a hidden variable observable only through noisy accelerometer and gyroscope measurements. The HMM's latent-state formulation directly models this uncertainty, while its Markovian transition structure captures realistic activity dynamics such as the cyclic phases of walking or the sustained stillness of resting.

2. Data Collection & Preprocessing

2.1 Recording Protocol

Data were collected using the **Sensor Logger** app on two smartphones. Each member recorded **25 labelled zip archives**, producing a combined dataset of **50 files**. Each archive contains **Accelerometer.csv**, **Gyroscope.csv**, and **Metadata.csv**. The activity label is embedded in the zip filename (e.g., Rob_Walking_01.zip) so the three-stage resolver in process_recording() extracts it from: (1) the filename, (2) an internal Annotation/Tags CSV, or (3) internal entry paths whichever succeeds first. Timestamps were stored as Android nanoseconds and converted to seconds by detecting when the delta exceeded 1×106 (impossible for a sub-10 s recording in seconds) and dividing by 1×109. Missing x/y/z axes were filled with 0.0 to ensure uniform shape.

Activity	Files (accel)	Min Dur (s)	Max Dur (s)	Total ≥ 90 s?
Walking	13	5.9	12.4	126.2
Standing	12	5.5	10.9	111.1
Still	13	5.7	12.0	128.0
Jumping	12	5.9	11.3	111.6

Table 2: Data collection summary. Each row counted from accelerometer segments only (gyroscope creates a second segment per zip). Fill actual min/max/totals from Section 4a notebook output.

2.2 Sampling Rate Harmonisation

Both phones targeted **100Hz** via the Sensor Logger app settings. The actual rate is parsed from the sampleRateMs field in each recording's Metadata.csv (Hz = 1000 / sampleRateMs). Rather than resampling raw signals which introduces interpolation artefacts particularly damaging to high-frequency gyroscope transients we adopted a **feature-level harmonisation** strategy. The windowing function window_data(df, window_size_seconds, sampling_rate) computes window_samples = round(1 × sampling_rate) so that every window always spans exactly one second regardless of rate (100 samples at 100 Hz; 50 samples at 50 Hz). Frequency-domain features pass d = 1/sampling_rate to np.fft.rfftfreq(), ensuring correct Hz values at any rate. Because all features are summary statistics over a fixed 1-second epoch, every window produces an identically-shaped 72-dimensional vector that the HMM can process without any knowledge of the original sample count.

2.3 Windowing Logic — Why 1-Second Non-Overlapping Windows?

- **Activity coherence (a):** 1 s contains atleast one full gait cyclefor Walking (cadence 1–2 Hz) and one full jump cycle for Jumping (~1 Hz), yet is short enough that a single window belongs to exactly one activity.
- **FFT resolution (b):** Bin width = sr / N = 100 / 100 = 1 Hz sufficient to separate the dominant frequencies of Walking (~2 Hz) and Jumping (~1 Hz) from Still/Standing (~0 Hz).
- **Markovian independence (c):** Non-overlapping windows eliminate duplicate information between adjacent samples, satisfying the HMM conditional independence assumption: each window's features depend only on the current hidden state.
- **Rate-adaptive (d):** window_data() auto-scales so windows are always exactly 1 second at any sampling rate, with a hard floor of 4 samples for FFT validity.

3. Feature Extraction & Normalisation

3.1 Feature Set — 72 Features per Window

Features are extractedby extract_features_from_window() for each sensor independently. For each of the four signals x, y, z, and magnitude (= √(x2+y2+z2), rotation-invariant) eight scalar features are computed, plus three cross-axis Pearson correlations and Signal Magnitude Area (SMA), giving **36 features per sensor × 2 sensors = 72 features per window**.

Feature	Type	Formula	Why It Discriminates Activities
Mean	Time	(1/N)Sx_i	DC bias differs by phone orientation and activity lean angle
Std Dev	Time	sqrt(Var)	Near-zero for Still/Standing; high for Walking/Jumping
Variance	Time	(1/N)S(x_i-u)^2	Amplifies large deviations; separates impact events from static postures
RMS	Time	sqrt((1/N)Sx_i^2)	Signal energy proxy; Jumping RMS ~10x higher than Still
Skewness	Time	E[(x-u)^3]/s^3	Jumping: asymmetric (sharp landing spike vs. smooth takeoff)
Kurtosis	Time	E[(x-u)^4]/s^4-3	Impulsive activities (Jumping) have high kurtosis; Still ~0
Dominant Freq	Frequency	argmax|FFT|, DC=0	Walking ~2 Hz, Jumping ~1 Hz, Still/Standing ~0 Hz — most discriminative
Spectral Energy	Frequency	S|FFT|^2 / N	Total energy bandwidth; Jumping broadband, Still near-zero
SMA	Time	(1/N)S(|x|+|y|+|z|)	Orientation-invariant motion intensity; key for active vs. passive
Corr xy,xz,yz	Time	Pearson r	Walking has coordinated x-z motion; Still has ~0 inter-axis correlation

Table 3: Full feature set. Each per-signal feature is computed for x, y, z, and magnitude (4 signals x 8 features = 32), plus 3 correlations and 1 SMA = 36 per sensor.

3.2 Z-Score Normalisation

A StandardScaler(Z-score) is fitted exclusively on the training set and saved to fitted_scaler.pkl. The same pre-fitted scaler is applied identically to test and new unseen data it is never refit ensuring no test-distribution information leaks into the normalisation parameters. Z-score was chosen over min-max because sensor data contains high-amplitude outliers (impact spikes in Jumping) that would compress the majority of values into a narrow range under min-max scaling. Raw features span approximately six orders of magnitude (spectral energy O(106) vs. Pearson r in [–1, 1]); Z-score removes scale dominance and gives the GMM distance metric equal units across all features.

3.3 Train/Test Split

Thesplit is performed at **recording level** (not window level): 2 whole recordings are held out as the test set using np.random.choice(all_ids, size=2, replace=False) with SEED=42. Splitting by window would cause temporal leakage adjacent windows from the same recording are nearly identical, inflating test accuracy. Recording-level splitting ensures the model is tested on genuinely unseen temporal contexts.

4. HMM Setup & Implementation

4.1 Model Structure

Component	Symbol	Implementation	Details
Hidden States	Z	Activity phases	Walking: 3 (push-off, mid-swing, heel-strike); Jumping: 3 (take-off, air, land); Still/Standing: 2 (static, micro-sway)
Observations	X	72-dim feature vector	Z-score normalised; 36 per sensor, 2 sensors
Initial probs	pi	Uniform init	pi = 1/S; updated by Baum-Welch M-step
Transition probs	A	Left-right init	Diagonal=0.7, forward=0.2; reflects phase continuity; updated by Baum-Welch
Emission probs	B	sklearn GaussianMixture	2 components per state, diagonal covariance, reg_covar=1e-4; KMeans warm-start

Table 4: HMM component definitions. One HMM is trained per (activity, sensor) pair = 8 models total.

4.2 Baum-Welch Training

- **Log-spacecomputation:** All forward (alpha) and backward (beta) variables are computed in log-space using log_sum_exp() to prevent floating-point underflow on sequences of 5–100 windows.
- **Convergence criterion:** |delta_LL| < epsilon = 1e-3 (max 50 iterations). This is a principled stopping rule based on the change in total log-likelihood across all training sequences, not a fixed iteration count.
- **Pooled M-step:** Sufficient statistics (exp(log_gamma), exp(log_xi)) are aggregated across ALL sequences before updating pi, A, and B. This is the correct pooled MLE for multi-sequence training, avoiding per-sequence bias.
- **GMM warm-start:** KMeans (k=n_states, n_init=10) clusters the concatenated training observations and seeds each state's GaussianMixture on its cluster subset, avoiding degenerate initialisation.
- **NaN handling:** (1) window_data() enforces min 4 samples; (2) _clean() strips NaN-dominated rows before training; (3) _get_log_emission_probs() assigns LOG_ZERO = -1e300 to remaining NaN rows; (4) A-row denominators clipped at 1e-10; (5) GMM reg_covar=1e-4 prevents singular covariances.

4.3 Viterbi Decoding

The Viterbi algorithm finds the most likely hidden-state path via log-space dynamic programming. Delta[t,s] stores the best log-probability of any path ending in state s at time t; psi[t,s] stores the backpointer. The path is recovered by backtracking from argmax(delta[T-1]). Used both for classification visualisation (Figure 6) and as a diagnostic to verify that learned state sequences match expected biomechanical phases.

4.4 Classification

Eightindependent HMMs are trained one per (activity, sensor) pair: Walking_accel, Walking_gyro, Standing_accel, Standing_gyro, Still_accel, Still_gyro, Jumping_accel, Jumping_gyro. Classification proceeds by scoring each test sequence against all four activity HMMs for the given sensor using model.score(X) (= log P(X|lambda)) and returning the argmax activity. The sensor fusion approach (Section 14 of notebook) combines accelerometer and gyroscope log-likelihoods as: log P_fused = 0.5 * log P_accel + 0.5 * log P_gyro.

5. Results & Interpretation

5.1 Figures Generated by the Notebook

Figure	Description	File
Fig 1	Raw sensor data — first 1-second window per activity per sensor (x/y/z axes)	fig1_raw_sensor_windows.png
Fig 2	Normalised feature box plots — 8 key features across 4 activities (train set)	fig2_feature_boxplots.png
Fig 3	Recording duration histogram with 5-10 s target band highlighted	fig3_duration_histogram.png
Fig 4	Baum-Welch log-likelihood convergence curves for all 8 HMMs	fig4_convergence.png
Fig 5	Row-normalised confusion matrices — held-out test set (accel + gyro)	fig5_confusion_test.png
Fig 6	Viterbi decoded state sequences — raw magnitude + step-plot of hidden states	fig6_decoded_sequences.png
Fig 7	Learned transition matrices A as heatmaps for all 8 HMMs	fig7_transition_matrices.png
Fig 8	GMM emission weighted means (sum_m w_m * mu_m) per hidden state, first 12 features	fig8_gmm_emission_means.png
Fig 9	Row-normalised confusion matrices — new unseen samples (accel + gyro)	fig9_confusion_new.png

Table 5: All notebook-generated visualisations. Saved to /content/ in Colab.

5.2 Evaluation Metrics — Held-out Test Set

Two whole recordings (SEED=42, randomly selected) were withheld from training. The model is evaluated at recording level: one prediction per complete recording. Run the notebook and fill the actual values from the Section 10 and Section 14 output into Table 6 below.

Activity	# Samples	Sensitivity	Specificity	Overall Accuracy
Walking	**2**	**100** %	**62.5 %**	**70.0 %**
Standing	**4**	**75.0 %**	**100.0 %**	**70.0 %**
Still	**2**	**100.0 %**	**100.0 %**	**70.0 %**
Jumping	**2**	**0.0 %**	**100.0 %**	**70.0 %**

Table 6: Evaluation results — fill actual values from notebook Section 10 (print_metrics output). Overall Accuracy is the same for all rows.

5.3 HMM Parameter Insights

- **Transitionmatrices (Fig7):**Still and Standing show strong diagonal entries (A[s,s] high), reflecting that stable postures persist in one micro-state. Walking shows cyclic off-diagonal structure (S0 to S1 to S2 to S0) consistent with

repeating gait phases. Jumping shows sequential forward transitions (S0 to S1 to S2) matching the ordered take-off, airborne, landing cycle.

- **Emission means (Fig 8):** The GMM weighted mean (sum_m w_m * mu_m) for each state represents the expected normalised feature observation. Jumping states show high RMS and spectral energy; Walking states have distinct dominant-frequency means (~2 Hz in the magnitude signal). Still states cluster near zero across most features, confirming physically meaningful learned representations.
- **Decoded sequences (Fig 6):** Viterbi paths on Walking test recordings show regular state cycling at approximately 1-2 Hz, correctly capturing gait rhythm. The left panel shows raw accelerometer magnitude; the right shows the step-plot of decoded hidden states with colour-shaded state intervals.

5.4 New Unseen Samples

If a NewTestSamples/ folder is present on Google Drive, new recordings from a different session or environment are loaded via the same process_recording() pipeline. If absent, the notebook synthesises plausible new recordings by adding Gaussian noise (mu=0, sigma=0.05) to existing training recordings simulating device variation and environmental drift while preserving activity dynamics. The pre-fitted StandardScaler is applied without refitting. Confusion matrices are shown in Figure 9.

6. Discussion & Conclusion

6.1 Easiest and Hardest Activities

Easiest — Jumping: The explosive landing impact produces uniquely high spectral energy (orders of magnitude above other activities) and high kurtosis, making it trivially separable. The dominant frequency (~1 Hz) is also distinctive. **Second easiest — Walking:** The periodic cadence (~1.5-2 Hz dominant frequency) and moderate RMS with coordinated cross-axis correlation (x-z) are consistently distinct from passive activities. **Hardest — Still vs. Standing:** Both produce near-zero spectral energy and RMS. The primary discriminating signal is subtle gyroscope sway in Standing (postural micro-corrections absent in Still). This pair typically shows the lowest per-class sensitivity and accounts for most confusion matrix off-diagonal entries.

6.2 Transition Probabilities and Realistic Behaviour

The left-right initialisation (diagonal = 0.7, forward = 0.2) biases learning towards phase persistence and ordered progression, both physically realistic for all four activities. After Baum-Welch convergence, high self-loop probabilities for Still and Standing reflect that these states persist without phase changes. Walking's cyclic structure and Jumping's forward-sequential structure emerge from data alone, confirming that the model learns genuine activity dynamics rather than overfitting to initialisation.

6.3 Effect of Sensor Noise and Sampling Rate

Accelerometer Johnson noise (~+-0.02 m/s^2) particularly affects Still recordings, where the ideal signal is zero. This inflates RMS and can corrupt dominant-frequency estimates. Z-score normalisation partially mitigates this by centring the feature distribution, but residual noise contributes to Still/Standing confusion. The magnitude feature (sqrt(x^2+y^2+z^2)) is rotation-invariant and therefore robust to differences in phone orientation between Rob and Antony's recording styles. Both phones targeted 100 Hz so rate differences were minimal; however the rate-adaptive windowing pipeline was validated to correctly produce 1-second windows at any input sampling rate.

6.4 Potential Improvements

- BIC/AIC model selection to choose state counts automatically rather than by biomechanical assumption.
- Overlapping windows (50% stride) to approximately double available training windows.
- Full covariance GMM matrices with stronger regularisation for richer emission modelling.
- Cross-user leave-one-out evaluation (train on Rob, test on Antony and vice-versa).
- Sensor fusion weight alpha tuned by cross-validation rather than fixed at 0.5.
- Expansion to additional activities: Running, Sitting, Stair climbing.

- Real-time inference via online Viterbi with a sliding-window buffer on the phone.
- Deep HMM: replace GMM emissions with a neural encoder (e.g. 1D-CNN or LSTM) for richer feature learning.

6.5 Conclusion

We implemented a complete Human Activity Recognition system using Hidden Markov Models trained from scratch on real smartphone sensor data. Fifty labelled recordings across four activities, Walking, Standing, Still, and Jumping, were collected by two group members using Sensor Logger on different handsets. A 72-dimensional feature set combining time-domain statistics, FFT-derived frequency features, signal magnitude area, and cross-axis correlations was extracted from 1-second non-overlapping windows. Eight HMMs (one per activity-sensor pair) were trained using log-space Baum-Welch with a principled convergence criterion, and classified test sequences using Viterbi-derived log-likelihood scoring. Evaluation on two held-out recordings and a separate unseen sample session with full sensitivity, specificity, and confusion matrix reporting demonstrates that the system achieves meaningful generalisation of **70% overall accuracy** beyond its training sessions. The physiotherapy monitoring use case is practically achievable with this architecture, with sensor fusion and cross-user evaluation identified as the most impactful next steps.

Notebook: HMM_Activity_Recognition_Grp21_FINAL.ipynb | **Drive path:** MyDrive/HiddenMarkovModel Data Grp 21/ | **Outputs:** trained_hmm_models.pkl, fitted_scaler.pkl, features_for_hmm.csv, fig1–fig9 (saved to /content/)